

Veyrivo Lead Generation CRM

Software Requirements Specification

Product: Veyrivo Lead Generation CRM

Version: 1.0

Status: Production planning document

Date: August 2026

Primary users: Veyrivo sales, founders, business-development staff, project managers, and administrators.

The CRM will help Veyrivo discover, organize, qualify, follow up with, and convert prospects into software, AI, automation, ERP, e-commerce, and cloud-development clients.

1. Product Objectives

1.1 Primary Objectives

- Centralize companies, contacts, prospects, conversations, tasks, proposals, and deals.
- Improve the client-hunting process.
- Prioritize leads using configurable scoring and AI-assisted qualification.
- Prevent missed follow-ups.
- Track every opportunity from discovery to conversion.
- Provide clear sales and business-development analytics.
- Create a reusable CRM foundation that can later become a multi-tenant SaaS product.

1.2 Business Outcomes

- Faster lead response.
- Better sales visibility.
- Fewer missed opportunities.
- More consistent follow-ups.
- Improved proposal conversion.
- Better understanding of profitable services and industries.
- Reusable portfolio project for Veyrivo client presentations.

1.3 Out of Scope for Version 1

- Full accounting system.
- Full email provider replacement.
- Uncontrolled bulk spam sending.
- Complete marketing automation suite.
- Complex payroll and HR management.
- Automatic scraping of restricted or private websites.
- Fully autonomous AI sales decisions.

2. Recommended Technology Stack

2.1 Frontend

- Next.js with App Router.
- TypeScript with strict mode.
- Tailwind CSS.
- shadcn/ui.
- TanStack Query for client-side server-state management where needed.
- React Hook Form.
- Zod for client and server validation.
- Recharts or another lightweight charting library.
- Motion for carefully controlled interface animations.

2.2 Backend

- NestJS.
- REST API with OpenAPI/Swagger.
- Prisma ORM.
- PostgreSQL.
- Redis for caching, rate limiting, queues, and background jobs.
- BullMQ for asynchronous processing.
- WebSocket or Server-Sent Events only where real-time updates are genuinely required.

2.3 Infrastructure

- Docker.
- GitHub Actions.
- Managed PostgreSQL.
- Managed Redis.
- Object storage for proposal files and attachments.
- Sentry or equivalent error monitoring.
- Structured JSON logging.
- Health checks and metrics.
- Separate development, staging, and production environments.

2.4 Why Prisma over TypeORM

Use **Prisma** for this CRM unless your team already has deep TypeORM expertise. Prisma is a good fit because:

- The schema is easy to review.
- Type-safe database access reduces common query mistakes.
- Migrations are explicit.
- Relations and indexes are easy to express.
- It integrates well with NestJS and PostgreSQL.
- It supports PostgreSQL-specific index types and database features.

Choose TypeORM instead if your team strongly prefers decorator-based entities, relies heavily on an existing TypeORM codebase, or requires patterns already standardized around TypeORM.

3. User Roles

Super Administrator: Manage workspaces, users, roles, services, settings, and integrations. View all records. Configure scoring rules. View audit logs. Manage system-level settings.

Sales Manager: View and manage all assigned sales data. Assign leads. Review pipeline performance. Approve proposals. View team analytics.

Sales Representative: View assigned companies, contacts, leads, deals, tasks, notes, and proposals. Update lead status. Schedule follow-ups. Create proposals. Cannot access restricted administration settings.

Marketing User: Manage campaigns, segments, templates, and consent-based subscribers. View campaign performance. Cannot access confidential deal information unless permitted.

Project Manager: View converted deals and client requirements. Access project handover information. Manage discovery and onboarding data.

Viewer: Read-only access to permitted records.

Use organization-level permissions from the beginning so the system can later support multiple companies and SaaS tenants.

4. Core Modules

4.1 4.1 Authentication and Authorization

Email and password authentication; optional OAuth login; password reset; email verification; multi-factor authentication later; session management; role-based access control; organization/workspace isolation; permission checks at controller and service levels; audit logging for sensitive actions.

4.2 4.2 Dashboard

Display: total leads, new leads this week, hot/warm/cold leads, open deals, pipeline value, proposals sent, conversion rate, follow-ups due today, overdue activities, leads by service, leads by source, monthly revenue or estimated opportunity value, sales funnel chart, and recent activity feed.

4.3 4.3 Companies

Manage: business name, website, industry, company size, country and city, contact details, lead source, assigned owner, tags, notes, related contacts, related opportunities, related proposals, and interaction history.

4.4 4.4 Contacts

Manage: name, job title, email, phone, WhatsApp number, LinkedIn profile, decision-maker status, communication preferences, consent status, company relationship, notes, and activities.

4.5 4.5 Leads

A lead is a potential sales opportunity before it becomes a qualified deal.

Lead Lifecycle Flow:

New → Contacted → Qualification → Discovery Scheduled → Qualified → Proposal Sent → Negotiation → Won → Lost

Additional statuses: Unqualified, Nurture, On Hold, Duplicate, Do Not Contact.

4.6 4.6 Deals and Opportunities

A deal represents a qualified commercial opportunity. Manage: deal title, company, primary contact, service, estimated value, currency, probability, expected close date, sales stage, owner, competitors, requirements, proposal, notes, activity timeline, lost reason, and conversion date.

4.7 4.7 Activities and Follow-ups

Support: calls, emails, meetings, WhatsApp conversations, notes, tasks, reminders, proposal follow-ups, and discovery actions. Each activity includes: type, title, description, due date, completion status, assigned user, related company, related contact, and related lead or deal.

4.8 4.8 Proposals

Support proposal templates, client information, scope of work, deliverables, timeline, milestones, payment schedule, terms, expiration date, version history, PDF generation, internal approval, and status tracking.

Proposal Statuses:

Draft → Internal Review → Sent → Viewed → Accepted / Rejected / Expired

4.9 4.9 Services and Industries

Create configurable service records: Custom web development, Mobile application development, AI applications, AI chatbots, Business automation, ERP systems, E-commerce, API development, AI document processing, Cloud deployment, Technical support.

Create industry records: Retail, Wholesale, Automotive, Education, Healthcare, Logistics, Professional services, Real estate, Manufacturing, Startups.

4.10 4.10 AI Qualification

The AI service may: Summarize lead requirements; detect requested service; estimate urgency; identify missing information; suggest lead temperature; recommend next action; extract budget and timeline when explicitly provided; suggest discovery questions.

The AI must not: Automatically reject leads without human review; invent a budget; claim the lead is fraudulent without evidence; send messages without approved automation rules; make legally or financially sensitive decisions.

Use structured model output so the result matches a defined JSON schema.

4.11 4.11 Campaigns and Email

Support permission-based email campaigns: contact segments, campaign templates, personalization variables, scheduling, delivery status, bounces, complaints, unsubscribe management, suppression lists, consent history, and domain authentication status. Avoid presenting this module as a spam tool.

4.12 4.12 Import and Export

Support: CSV import, CSV export, duplicate detection, column mapping, import preview, validation errors, import history, permission-based export, and export audit logs.

4.13 4.13 Notifications

Support: in-app notifications, email notifications, optional WhatsApp notifications, daily follow-up digest, new hot-lead alerts, proposal-view alerts, overdue task reminders, and assignment notifications.

5. Database Design

Use UUID primary keys, UTC timestamps, soft deletion where appropriate, and organization-level ownership.

5.1 5.1 organizations

Column	Type	Notes
id	UUID	Primary key
name	VARCHAR(150)	Required
slug	VARCHAR(100)	Unique
logo_url	TEXT	Optional
timezone	VARCHAR(60)	Default UTC
currency	VARCHAR(3)	Default PKR or configurable
status	ENUM	Active, suspended
created_at	TIMESTAMPTZ	Required
updated_at	TIMESTAMPTZ	Required

5.2 5.2 users

Column	Type	Notes
id	UUID	Primary key
email	VARCHAR(320)	Unique
password_hash	TEXT	Never store plaintext
first_name	VARCHAR(80)	Required
last_name	VARCHAR(80)	Required
avatar_url	TEXT	Optional
status	ENUM	Active, invited, suspended
last_login_at	TIMESTAMPTZ	Optional
created_at	TIMESTAMPTZ	Required
updated_at	TIMESTAMPTZ	Required

5.3 5.3 organization_members

Column	Type	Notes
id	UUID	Primary key
organization_id	UUID	FK
user_id	UUID	FK
role_id	UUID	FK
joined_at	TIMESTAMPTZ	Required
created_at	TIMESTAMPTZ	Required

Unique constraint: organization_id + user_id

5.4 5.4 roles

Column	Type	Notes
id	UUID	Primary key
organization_id	UUID	Nullable for system roles
name	VARCHAR(80)	Required
description	TEXT	Optional
created_at	TIMESTAMPTZ	Required

5.5 5.5 permissions

Column	Type	Notes
id	UUID	Primary key
code	VARCHAR(120)	Unique
description	TEXT	Optional

5.6 5.6 role_permissions

Column	Type	Notes
role_id	UUID	FK
permission_id	UUID	FK

Composite primary key: role_id + permission_id

5.7 5.7 companies

Column	Type	Notes
id	UUID	Primary key
organization_id	UUID	FK
name	VARCHAR(200)	Required
slug	VARCHAR(220)	Unique per organization
website	TEXT	Optional
domain	VARCHAR(255)	Optional
industry_id	UUID	FK
company_size	ENUM	Startup, small, medium, enterprise
country	VARCHAR(100)	Optional
city	VARCHAR(100)	Optional
address	TEXT	Optional
phone	VARCHAR(40)	Optional
source_id	UUID	FK
owner_id	UUID	FK to user
status	ENUM	Prospect, customer, inactive
description	TEXT	Optional
created_at	TIMESTAMPTZ	Required
updated_at	TIMESTAMPTZ	Required
deleted_at	TIMESTAMPTZ	Soft delete

5.8 5.8 contacts

Column	Type	Notes
id	UUID	Primary key
organization_id	UUID	FK
company_id	UUID	FK
first_name	VARCHAR(80)	Required
last_name	VARCHAR(80)	Required
email	VARCHAR(320)	Optional
phone	VARCHAR(40)	Optional
what-sapp_phone	VARCHAR(40)	Optional
job_title	VARCHAR(120)	Optional
linkedin_url	TEXT	Optional
is_decision_maker	BOOLEAN	Default false
consent_status	ENUM	Unknown, opted_in, opted_out
consent_at	TIMESTAMPTZ	Optional
owner_id	UUID	FK
created_at	TIMESTAMPTZ	Required
updated_at	TIMESTAMPTZ	Required
deleted_at	TIMESTAMPTZ	Soft delete

5.9 5.9 leads

Column	Type	Notes
id	UUID	Primary key
organization_id	UUID	FK
company_id	UUID	Optional FK
contact_id	UUID	Optional FK
title	VARCHAR(200)	Required
description	TEXT	Optional
status	ENUM	New, contacted, qualified, nurture, lost
temperature	ENUM	Hot, warm, cold, unknown
score	SMALLINT	0–100
source_id	UUID	FK
service_id	UUID	FK
industry_id	UUID	FK
estimated_value	NUMERIC(14,2)	Optional
currency	VARCHAR(3)	Required
timeline	ENUM	Urgent, one_month, three_months, exploring
budget_range	ENUM	Low, medium, high, unknown
owner_id	UUID	FK
ai_summary	TEXT	Optional
ai_result	JSONB	Optional
qualified_at	TIMESTAMPTZ	Optional
last_contacted_at	TIMESTAMPTZ	Optional
next_follow_up_at	TIMESTAMPTZ	Optional
created_at	TIMESTAMPTZ	Required
updated_at	TIMESTAMPTZ	Required
deleted_at	TIMESTAMPTZ	Soft delete

5.10 5.10 lead_sources

Column	Type	Notes
id	UUID	Primary key
organization_id	UUID	FK
name	VARCHAR(100)	Website, LinkedIn, referral, etc.
type	ENUM	Organic, paid, referral, outbound, event
created_at	TIMESTAMPTZ	Required

5.11 5.11 services

Column	Type	Notes
id	UUID	Primary key
organization_id	UUID	FK
name	VARCHAR(150)	Required
slug	VARCHAR(180)	Unique
description	TEXT	Optional
is_active	BOOLEAN	Default true
created_at	TIMESTAMPTZ	Required

5.12 5.12 industries

Column	Type	Notes
id	UUID	Primary key
organization_id	UUID	FK
name	VARCHAR(120)	Required
slug	VARCHAR(140)	Unique
is_active	BOOLEAN	Default true
created_at	TIMESTAMPTZ	Required

5.13 5.13 pipeline_stages

Column	Type	Notes
id	UUID	Primary key
organization_id	UUID	FK
name	VARCHAR(100)	Required
slug	VARCHAR(120)	Unique
position	SMALLINT	Ordering
probability	SMALLINT	0-100
color	VARCHAR(20)	UI color
is_closed	BOOLEAN	Default false
is_won	BOOLEAN	Default false
created_at	TIMESTAMPTZ	Required

5.14 5.14 deals

Column	Type	Notes
id	UUID	Primary key
organization_id	UUID	FK
lead_id	UUID	Optional FK
company_id	UUID	FK
contact_id	UUID	Optional FK
title	VARCHAR(200)	Required
stage_id	UUID	FK
owner_id	UUID	FK
value	NUMERIC(14,2)	Required
currency	VARCHAR(3)	Required
expected_close_date	DATE	Optional
won_at	TIMESTAMPTZ	Optional
lost_at	TIMESTAMPTZ	Optional
lost_reason	VARCHAR(200)	Optional
created_at	TIMESTAMPTZ	Required
updated_at	TIMESTAMPTZ	Required

5.15 5.15 activities

Column	Type	Notes
id	UUID	Primary key
organization_id	UUID	FK
company_id	UUID	Optional FK
contact_id	UUID	Optional FK
lead_id	UUID	Optional FK
deal_id	UUID	Optional FK
assigned_to_id	UUID	FK
created_by_id	UUID	FK
type	ENUM	Call, email, meeting, note, task, message
title	VARCHAR(200)	Required
description	TEXT	Optional
due_at	TIMESTAMPTZ	Optional
completed_at	TIMESTAMPTZ	Optional
status	ENUM	Pending, completed, cancelled
created_at	TIMESTAMPTZ	Required
updated_at	TIMESTAMPTZ	Required

5.16 5.16 proposals

Column	Type	Notes
id	UUID	Primary key
organization_id	UUID	FK
deal_id	UUID	FK
version	INTEGER	Required
title	VARCHAR(200)	Required
status	ENUM	Draft, review, sent, viewed, accepted, rejected, expired
content	JSONB	Structured proposal content
total_value	NUMERIC(14,2)	Optional
currency	VARCHAR(3)	Required
valid_until	DATE	Optional
sent_at	TIMESTAMPTZ	Optional
viewed_at	TIMESTAMPTZ	Optional
accepted_at	TIMESTAMPTZ	Optional
created_by_id	UUID	FK
created_at	TIMESTAMPTZ	Required
updated_at	TIMESTAMPTZ	Required

5.17 5.17 tags

Column	Type	Notes
id	UUID	Primary key
organization_id	UUID	FK
name	VARCHAR(80)	Required
color	VARCHAR(20)	Optional
created_at	TIMESTAMPTZ	Required

Join tables: company_tags, contact_tags, lead_tags, deal_tags. Each contains entity_id and tag_id using composite primary keys.

5.18 5.18 attachments

Column	Type	Notes
id	UUID	Primary key
organization_id	UUID	FK
uploaded_by_id	UUID	FK
company_id	UUID	Optional FK
lead_id	UUID	Optional FK
deal_id	UUID	Optional FK
proposal_id	UUID	Optional FK
storage_key	TEXT	Required
file_name	VARCHAR(255)	Required
mime_type	VARCHAR(120)	Required
size_bytes	BIGINT	Required
created_at	TIMESTAMPTZ	Required

5.19 5.19 audit_logs

Column	Type	Notes
id	UUID	Primary key
organization_id	UUID	FK
actor_id	UUID	Optional FK
action	VARCHAR(100)	Created, updated, deleted, exported
entity_type	VARCHAR(80)	Lead, deal, proposal
entity_id	UUID	Related record
old_values	JSONB	Optional
new_values	JSONB	Optional
ip_address	INET	Optional
user_agent	TEXT	Optional
created_at	TIMESTAMPTZ	Required

6. Relationships

```

Organization
|-- Users through OrganizationMembers
|-- Companies ---+--- Contacts
|                   |-- Leads
|                   |-- Deals
|                   `-- Activities
|-- Contacts ---+--- Leads
|                   `-- Deals
|-- Leads -----+--- Company, Contact, Service, Industry, Source, Activities, Deal
|-- Deals -----+--- PipelineStage, Activities, Proposals
|-- Proposals --`-- Attachments
`-- Services, Industries, Tags, AuditLogs

```

All organization-owned records must include `organization_id` to prevent cross-tenant data access.

7. Prisma Schema Direction

```

model Lead {
  id                String    @id @default(uuid()) @db.Uuid
  organizationId    String    @db.Uuid
  companyId         String?   @db.Uuid
  contactId        String?   @db.Uuid
  serviceId        String?   @db.Uuid
  industryId       String?   @db.Uuid
  sourceId         String?   @db.Uuid
  ownerId          String?   @db.Uuid

  title            String    @db.VarChar(200)
  description       String?
  status           LeadStatus @default(NEW)
  temperature      LeadTemperature @default(UNKNOWN)
  score            Int        @default(0)
  estimatedValue   Decimal?   @db.Decimal(14, 2)
  currency         String    @default("PKR") @db.VarChar(3)
  aiSummary        String?
  aiResult         Json?
  nextFollowUpAt   DateTime?
  lastContactedAt  DateTime?
}

```

```

qualifiedAt      DateTime?

createdAt        DateTime @default(now())
updatedAt        DateTime @updatedAt
deletedAt        DateTime?

organization      Organization @relation(fields: [organizationId], references: [id])
company          Company? @relation(fields: [companyId], references: [id])
contact          Contact? @relation(fields: [contactId], references: [id])
service          Service? @relation(fields: [serviceId], references: [id])
industry         Industry? @relation(fields: [industryId], references: [id])
source           LeadSource? @relation(fields: [sourceId], references: [id])
owner            User? @relation(fields: [ownerId], references: [id])

activities        Activity[]
deals             Deal[]

@@index([organizationId, status])
@@index([organizationId, temperature])
@@index([organizationId, ownerId, status])
@@index([organizationId, nextFollowUpAt])
@@index([organizationId, serviceId])
@@index([organizationId, createdAt])
@@index([organizationId, score])
@@index([email])
@@map("leads")
}

enum LeadStatus {
  NEW
  CONTACTED
  QUALIFICATION
  QUALIFIED
  NURTURE
  LOST
}

enum LeadTemperature {
  HOT
  WARM
  COLD
  UNKNOWN
}

```

8. Indexing Strategy

8.1 Required Indexes

- **organization_members:** unique (organization_id, user_id)
- **companies:** index (organization_id, name), index (organization_id, owner_id), index (organization_id, industry_id), index (organization_id, status)
- **contacts:** index (organization_id, company_id), index (organization_id, email), index (organization_id, owner_id), index (organization_id, consent_status)
- **leads:** index (organization_id, status), index (organization_id, temperature), index (organization_id, owner_id, status), index (organization_id, service_id), index (organization_id, source_id), index (organization_id, next_follow_up_at), index (organization_id, created_at), index (organization_id, score)
- **deals:** index (organization_id, stage_id), index (organization_id, owner_id), index (organization_id, expected_close_date)

- **activities:** `index (organization_id, assigned_to_id, status), index (organization_id, due_at), index (organization_id, lead_id)`
- **proposals:** `index (organization_id, deal_id), index (organization_id, status), index (organization_id, valid_until)`
- **audit_logs:** `index (organization_id, created_at), index (organization_id, entity_type, entity_id)`

8.2 Search Indexes & Query Performance Rules

For company and contact search, begin with PostgreSQL full-text search or trigram indexes. For a larger CRM, consider Meilisearch, Typesense, or OpenSearch. Do not run unbounded `ILIKE '%term%'` queries.

- Always filter by `organization_id`. Select only required fields.
- Paginate every list endpoint. Use cursor pagination for large activity timelines.
- Avoid N+1 queries. Use database joins or carefully batched relation queries.
- Inspect slow queries with `EXPLAIN ANALYZE`.

9. API Design

Versioned REST APIs: `/api/v1/auth`, `/api/v1/organizations`, `/api/v1/users`, `/api/v1/companies`, `/api/v1/contacts`, `/api/v1/leads`, `/api/v1/deals`, `/api/v1/activities`, `/api/v1/proposals`, `/api/v1/services`, `/api/v1/industries`, `/api/v1/tags`, `/api/v1/analytics`, `/api/v1/integrations`.

9.1 Lead Endpoints

```
POST    /api/v1/leads
GET      /api/v1/leads
GET      /api/v1/leads/:id
PATCH   /api/v1/leads/:id
DELETE   /api/v1/leads/:id
POST     /api/v1/leads/:id/qualify
POST     /api/v1/leads/:id/convert
POST     /api/v1/leads/:id/activities
POST     /api/v1/leads/import
GET      /api/v1/leads/export
```

9.2 API Requirements & Sample Response

Use DTOs, validate input, pagination, consistent error responses, authorization on every resource, request IDs, rate limits, OpenAPI docs, versioning.

```
{
  "data": {
    "id": "lead-id",
    "title": "ERP inquiry",
    "status": "QUALIFIED",
    "score": 86
  },
  "meta": {
    "requestId": "request-id"
  }
}
```

10. Backend Architecture

```
src/  
  app.module.ts  
  common/ (decorators, guards, interceptors, filters, pipes, types)  
  config/ | database/ | auth/ | users/ | organizations/  
  companies/ | contacts/ | leads/ | deals/ | activities/  
  proposals/ | services/ | industries/ | tags/  
  notifications/ | integrations/ | ai/ | analytics/ | audit/ | health/
```

10.1 NestJS Production Configuration

Enable: ValidationPipe with whitelist, forbidNonWhitelisted, transform, Helmet, CORS allowlist, request size limits, rate limiting, authentication guards, RBAC/ABAC, global exception filters, structured logging, API versioning, health endpoints, graceful shutdown, and database connection monitoring.

11. Security Requirements

11.1 Authentication & Authorization

- Hash passwords with Argon2id or bcrypt.
- Use secure, HTTP-only cookies for web sessions. Rotate refresh tokens. Expire inactive sessions.
- Require email verification; add MFA for administrators.
- Check all three layers: Authenticated user → Organization membership → Permission for requested action. Do not rely only on frontend route protection.

11.2 Data Protection & API Security

- Encrypt data in transit (HTTPS only) and at rest. Separate secrets per environment.
- Rate-limit login, import, export, AI, and email endpoints.
- Validate uploaded file type and size. Scan uploaded files where appropriate.
- Prevent SSRF in website/domain enrichment. Sanitize user-generated HTML.
- Add idempotency keys for duplicate-prone operations. Log exports and destructive actions.

11.3 AI Security

- Treat lead messages as untrusted input. Use fixed system instructions.
- Do not allow lead text to override qualification rules.
- Validate model output against a schema. Add confidence thresholds.
- Require human review for uncertain results.
- Store model, prompt version, and result for traceability.

12. SEO Strategy

12.1 Public SEO Pages

Private CRM pages (/dashboard, /login, /admin, /settings) are excluded from indexing. Public indexable pages include: /, /services/custom-software, /services/ai-solutions, /services/ai-chatbots, /services/business-automation, /services/erp-development, /services/ecommerce-development, /industries/*, /work, /insights, /contact.

12.2 Next.js Metadata Example

```
import type { Metadata } from "next";

export const metadata: Metadata = {
  title: "AI Lead Generation CRM | Veyrivo Technologies",
  description: "Veyrivo Technologies builds intelligent CRM, AI, automation, ERP, and custom software solutions for growing businesses.",
  alternates: {
    canonical: "https://veyrivo.tech/services/lead-generation-crm",
  },
  openGraph: {
    title: "AI Lead Generation CRM | Veyrivo Technologies",
    description: "Organize leads, automate follow-ups, and convert more business opportunities.",
    type: "website",
  },
};
```

13. Performance and Speed

13.1 Frontend & API Optimization

- Use Server Components by default; Client Components only for interactivity.
- Use next/image, SVG icons, self-hosted fonts with font-display: swap.
- Lazy-load charts; use skeletons for slow data.
- Cache services, industries, and configuration data. Cache dashboard aggregates for short periods using Redis.
- Move email, imports, AI calls, and exports to BullMQ background queues.

13.2 Performance Targets

- **LCP:** under 2.5 seconds on key public pages.
- **INP:** under 200 milliseconds.
- **CLS:** under 0.1.
- **API p95 latency:** under 500ms for common reads, under 1s for normal writes.
- **Dashboard first load:** under 2 seconds on a normal connection.

14. Background Jobs

Use BullMQ with Redis for: AI lead qualification, email sending, email campaign scheduling, CSV imports, data enrichment, proposal PDF generation, daily follow-up reminders, analytics aggregation, notification delivery, and data exports.

Job Requirements: Retry with exponential backoff, idempotent processing, dead-letter queue, job status tracking, maximum retry count, error logging, manual retry from admin, and timeout handling.

15. Frontend Pages

Routes: /login, /forgot-password, /dashboard/overview, /dashboard/leads, /dashboard/leads/[id], /dashboard/companies, /dashboard/companies/[id], /dashboard/contacts, /dashboard/deals, /dashboard/pipeline, /dashboard/activities, /dashboard/proposals, /dashboard/campaigns, /dashboard/analytics, /dashboard/settings/*.

Main Dashboard Components: KPI cards, Lead funnel, Deal pipeline, Follow-up calendar, Recent activity timeline, Lead-source chart, Service-performance chart, Quick-add menu, Global search, Notifications panel.

16. Testing Strategy

- **Unit Tests:** Scoring rules, DTO validation, permission helpers, lead conversion rules, proposal calculations, status transitions.
- **Integration Tests:** Repositories, Auth/Authz, Lead creation/conversion, proposal lifecycle, tenant isolation, email/AI adapters.
- **E2E Tests:** Full user journeys from login to company creation, lead qualification, deal conversion, proposal dispatch, and data export.
- **Security & Load Tests:** BOLA/IDOR checks, rate limit validation, SQLi/XSS prevention, SSRF tests, dataset scaling tests.

17. CI/CD Pipeline

PR Execution Sequence: Install deps → Type check → ESLint → Unit/Integration tests → Build frontend & backend → Prisma migration validation → Dependency vulnerability scan.

Deployment Sequence: Merge to main → Build → DB migration → Deploy API & Frontend → Smoke tests → Monitor.

18. Observability

Structured JSON logs, Request ID, User/Org IDs in safe context, error monitoring (Sentry), API/DB latency metrics, BullMQ queue metrics, OpenAI token/cost tracking, email metrics, uptime checks.

Health Endpoints: GET /health, GET /health/live, GET /health/ready.

19. Environment Variables

```
NODE_ENV=production
DATABASE_URL=postgresql://...
DIRECT_DATABASE_URL=postgresql://...
REDIS_URL=redis://...
JWT_SECRET=...
SESSION_SECRET=...
OPENAI_API_KEY=...
OPENAI_MODEL=...
EMAIL_PROVIDER_API_KEY=...
EMAIL_FROM=hello@veyrivo.tech
STORAGE_ENDPOINT=...
STORAGE_BUCKET=...
STORAGE_ACCESS_KEY=...
```

```
STORAGE_SECRET_KEY=...  
NEXT_PUBLIC_APP_URL=https://crm.veyrivo.tech  
API_URL=https://api.veyrivo.tech
```

20. MVP Release Plan

1. **Phase 1: Foundation** — Project setup, auth, orgs/users, RBAC, DB migrations, layout, CI/CD, logging.
2. **Phase 2: CRM Core** — Companies, contacts, leads, lead statuses, services, sources, notes, activities, follow-ups.
3. **Phase 3: Sales Workflow** — Deals, pipeline stages, proposal builder, PDF generation, dashboard KPIs.
4. **Phase 4: AI and Automation** — AI lead summary, structured qualification, human review flow, notifications, n8n, background queues.
5. **Phase 5: Production Hardening** — Security/load testing, backups, monitoring, audit logs, disaster recovery, documentation.

21. Acceptance Criteria

The MVP is ready when: Users can register/be invited; tenant data is isolated; RBAC functions; companies/contacts can be created; leads can filter, search, and update through lifecycle stages; activities/follow-ups can be scheduled and completed; deals can be created from qualified leads; proposals can be generated and sent; dashboard metrics match DB records; AI outputs are schema-validated and human-reviewable; inputs are validated; pagination works; backups/rollbacks are active; public SEO metadata is present.

22. Final Recommendation

Build this CRM as a **single-tenant internal product first**, but include `organization_id` and permission boundaries from day one. That allows Veyrivo to use it immediately for client hunting while keeping a clear path toward a multi-tenant SaaS product.

Core Architectural Summary:

1. Use **NestJS + Prisma + PostgreSQL** for the API and data layer.
2. Use **Next.js + Tailwind CSS + shadcn/ui** for the frontend.
3. Use **Redis + BullMQ** for AI, email, imports, and scheduled jobs.
4. Use **structured OpenAI outputs** with human review.
5. Use organization-level authorization on every query.
6. Index according to real CRM query patterns.
7. Keep public marketing pages SEO-friendly and private CRM pages excluded from indexing.
8. Treat security, observability, backups, migrations, and testing as core features.